

Looping Structures and Arrays

By

Dr. G.N. Balaji

Associate Professor, SCOPE

Looping structure

➤ It is used to execute set of instructions in several time based on condition.

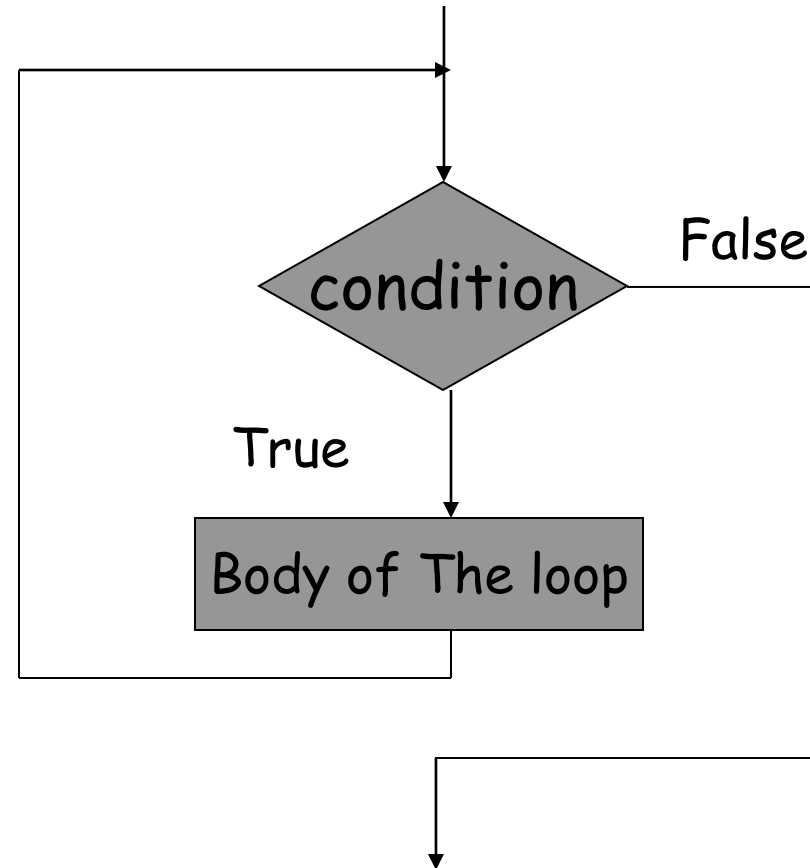
➤ **while**

- The while loop evaluates the test expression.
- If the test expression is true (nonzero), codes inside the body of while loop is executed. The test expression is evaluated again. The process goes on until the test expression is false.
- When the test expression is false, the while loop is terminated.

while loop

Syntax

```
.  
  
while(condition)  
{  
    .  
    Body of the loop;  
    .  
}
```



```
#include <stdio.h>
```

```
int main()
```

```
{
```

```
    printf( "Hello World\n");
```

```
    printf( "Hello World\n");
```

```
    printf( "Hello World\n");
```

```
    printf( "Hello World\n");
```

```
    printf( "Hello World\n");
```

```
    printf( "Hello World\n");
```

```
    printf( "Hello World\n");
```

```
    printf( "Hello World\n");
```

```
    printf( "Hello World\n");
```

```
    printf( "Hello World\n");
```

```
    return 0;
```

```
}
```

```
#include <stdio.h>
```

```
int main()
```

```
{
```

```
    // initialization expression
```

```
    int i = 1;
```

```
    // test expression
```

```
    while (i < 11)
```

```
    {
```

```
        printf( "Hello World\n");
```

```
        // update expression
```

```
        i++;
```

```
    }
```

```
    return 0;
```

```
}
```

Looping structure

➤ **do...while**

- The code block (loop body) inside the braces is executed once.
- Then, the test expression is evaluated. If the test expression is true, the loop body is executed again. This process goes on until the test expression is evaluated to 0 (false).
- When the test expression is false (nonzero), the do...while loop is terminated.

1. Write a C program to find the sum of first 10 natural numbers.

Expected Output :

The first 10 natural number is :

1 2 3 4 5 6 7 8 9 10

The Sum is : 55

2. Write a program in C to read 10 numbers from keyboard and find their sum and average.

Test Data :

Input the 10 numbers :

Number-1 :2

...

Number-10 :2

Expected Output :

The sum of 10 no is : 55

The Average is : 5.500000

1. Write a program in C to display the cube of the number upto given an integer.

Test Data :

Input number of terms : 5

Expected Output :

Number is : 1 and cube of the 1 is :1

Number is : 2 and cube of the 2 is :8

Number is : 3 and cube of the 3 is :27

Number is : 4 and cube of the 4 is :64

Number is : 5 and cube of the 5 is :125

2. Write a program in C to display the multiplication table of a given integer.

Test Data :

Input the number (Table to be calculated) : 15

Expected Output :

15 X 1 = 15

...

...

15 X 10 = 150

do...while Loop

Syntax

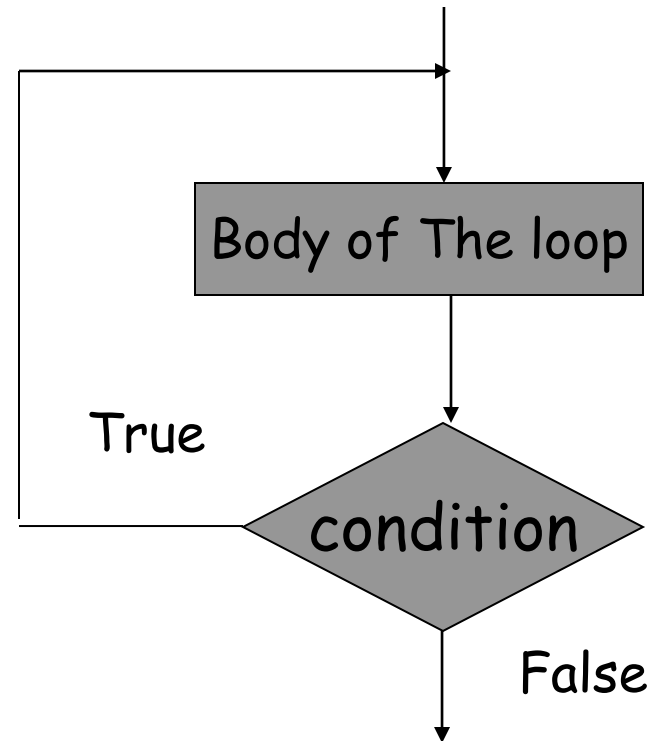
do

{

Body of the loop

}

while (condition);




```
#include <stdio.h>
```

```
int main()
{
    printf( "Hello World\n");
    printf( "Hello World\n");
    printf( "Hello World\n");
    printf( "Hello World\n");
    printf( "Hello World\n");
    printf( "Hello World\n");
    printf( "Hello World\n");
    printf( "Hello World\n");
    printf( "Hello World\n");

    return 0;
}
```

```
int main()
{
    int i = 2; // Initialization expression

    do
    {
        // loop body
        printf( "Hello World\n");

        // update expression
        i++;

    } while (i < 1); // test expression

    return 0;
}
```

Looping structure

➤ **for**

- The initialization statement is executed only once.
- Then, the test expression is evaluated. If the test expression is false (0), for loop is terminated. But if the test expression is true (nonzero), codes inside the body of for loop is executed and the update expression is updated.
- This process repeats until the test expression is false.

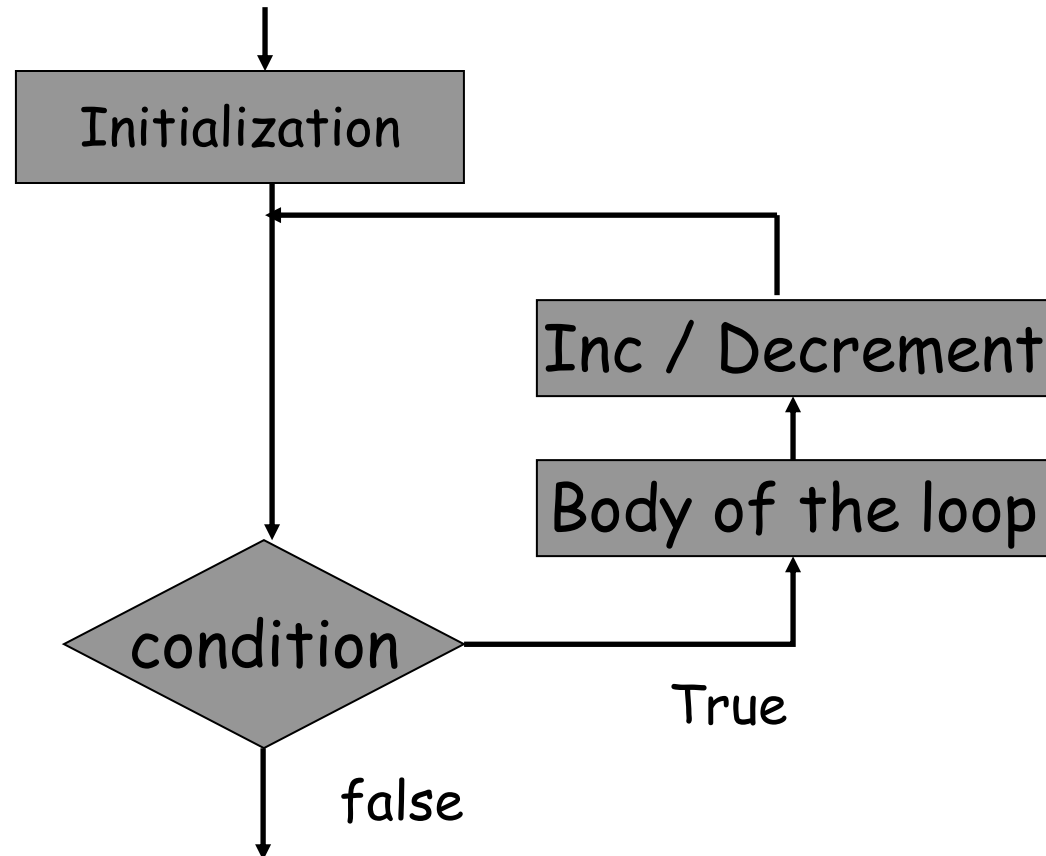
Syntax - for loop

for (initialization; condition; Increment/Decrement)

{

Body of the loop

}



Example: 6 (while)

```
int x = 2;
while(x < 100)
{
    printf("%d\n", x);
    x = x * 2;
}
```

Example: 7 (for)

```
int x;
for(x=2;x<100;x*=2)
    printf("%d\n", x);
```

Example: 8 (do-while)

```
do{
    printf("Enter 1 for yes, 0 for no :");
    scanf("%d", &input_value);
} while (input_value == 1);
```

Calculate factorial using while & for

```
#include<stdio.h>
#include<conio.h>
void main()
{
    int i=1,fact=1,n;
    printf("\nEnter the Number:");
    scanf("%d",&n);
    while(i<=n)
    {
        fact=fact*i;
        i++;          // i=i+1
    }
    printf("\n The value of %d! is:%d",n,fact);
    getch();
}
```

Output

Enter the Number:3

The value of 3! is: 6

```
for(i=1;i<=n;i++)
{
    fact=fact*i;
}
```

// add numbers until user enters zero using do-while

```
#include <stdio.h>
```

```
void main()
```

```
{
```

```
double number, sum = 0;
```

```
do {
```

```
    printf("Enter a number: ");
```

```
    scanf("%lf", &number);
```

```
    sum += number;
```

```
    } while(number != 0.0);
```

```
printf("Sum = %.2lf",sum);
```

```
}
```

Storage Classes

- These classes are used to determine the lifetime, visibility, memory location, and initial value of a variable.
- It can be classified into the following four types:
 - Automatic
 - Static
 - External
 - Register

static

- Keyword is static.
- The variable preceded with the keyword static known as static variable.
- Default initial value of the static integral variable is 0 otherwise null.
- Static local variables are visible only to the function/block in which they are defined.
- The visibility of the static global variable is to the entire program(file) in which it has declared.
- A same static variable can be declared many times but can be assigned at only one time.
- The variables defined as static specifier can hold their value between the multiple function calls.

static

```
#include<stdio.h>
static int a;
static float b;
static char c;
static char* s;
int main()
{
printf("Integer default value %d\n",a);
printf("Float default value %f\n",b);
printf("Char default value %d\n",c);
printf("Strings default value %s\n",s);
return 0;
}
```

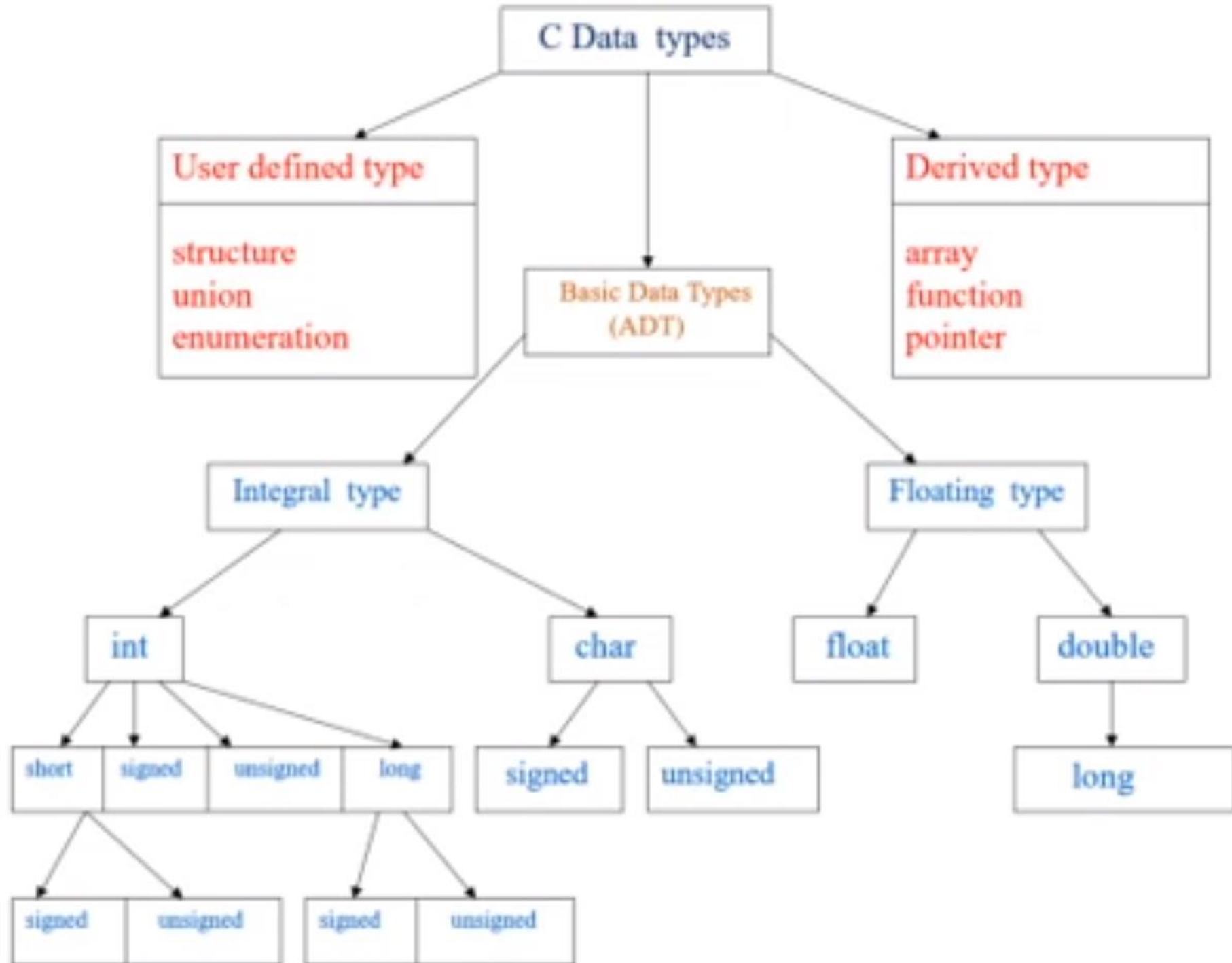
Output:

Integer default value 0

Float default value 0.000000

Char default value 0

Strings default value (null)



Arrays

- Collection of similar type of data items stored at contiguous memory locations.
- Arrays are the derived data type which can store the primitive type of data such as int, char, double, float, etc.
- It has the capability to store the collection of derived data types, such as pointers, structure, etc.
- Each data element can be randomly accessed by using its index (subscript) number.

Advantages of Array

- **Code Optimization:** Less code to access the data.
- **Ease of traversing:** By using for loop, we can retrieve the elements of an array easily.
- **Random Access:** We can access any element randomly using the array.

Disadvantages:

- **Fixed Size:** Whatever size, we define at the time of declaration of the array, we can't exceed the limit.

Characteristics of Arrays

- All the elements of an array share a common name called as array name
- The individual elements of an array are referred based on their position.
- The array index starts from 0.

Types of Array:

- One dimensional
- Multi dimensional

Syntax of 1D Array

`data_type udan[size];`

udan – user defined array name

size – size of an array

Declaration Ex.

`int a[5];`

Its an integer array.

Name of the array is a.

Size of the array is 5

Index starts from 0 to 4

If size of int is 2 bytes then it allocates 10 bytes memory in continuously.

Definition / Initialization

Ex.

`int a[5] = {10,20,30,25,20};`

or

`int a[]={10,20,30,25,20}`

a[0]	a[1]	a[2]	a[3]	a[4]
10	20	30	25	20

Input & Output for 1D Array

```
int ar[3], i;  
// Input data  
for(i=0;i<3;i++)  
    scanf("%d",&ar[i]);
```

```
//Output  
for(i=0;i<3;i++)  
    printf("%d\n",ar[i]);
```

//Input

11

22

15

ar[0]	ar[1]	ar[2]
11	22	15

//Output

11

22

15

Program to find mean or average

```
#include<stdio.h>
void main()
{
int i;
float ar[5],avg=0;
for(i=0;i<5;i++)
    scanf("%f",&ar[i]);
for(i=0;i<5;i++)
    avg=avg+ar[i];
avg=avg/5;
printf("%f",avg);
}
```

Input

15

25

8

7

18

i	avg=0	i<5	ar[i]
0	15	0<5 ->T	15
1	40	1<5 ->T	25
2	48	2<5 ->T	8
3	55	3<5 ->T	7
4	73	4<5 ->T	18
5	14.6	5<5 ->F	

Output

14.600000

- Write a program in C to read n number of values in an array and display it in reverse order.

Test Data :

Input the number of elements to store in the array :3

Input 3 number of elements in the array :

element - 0 : 2

element - 1 : 5

element - 2 : 7

Expected Output :

The values store into the array are :

2 5 7

The values store into the array in reverse are :

7 5 2

Two Dimensional Arrays

- The two-dimensional array can be defined as an array of arrays.
- The 2D array is organized as matrices which can be represented as the collection of rows and columns.
- Ex. Matrix, Tables

Syntax of 2D Array

`data_type udan[rowsize][colsize];`

udan – user defined array name

rowsize – size of the row

colsize – size of the column

Declaration Ex.

`int a[3][2];`

- Its an integer array.
- **Name of the array is a.**

- 3 rows and 2 columns
- **No. of elements 3 x 2 is 6**
- If size of int is 2 bytes then it allocates 12 bytes memory in continuously.
- **Index are**
(0,0) (0,1) -> 0th Row

Syntax of 2D Array

`data_type udan[rowsize][colsize];`

udan – user defined array name

rowsize – size of the row

colsize – size of the column

Declaration Ex.

`int a[3][2];`

- Its an integer array.
- Name of the array is a.

- 3 rows and 2 columns
- No. of elements 3 x 2 is 6
- If size of int is 2 bytes then it allocates 12 bytes memory in continuously.
- Index are
 - (0,0) (0,1) -> 0th Row
 - (1,0),(1,1) -> 1st Row
 - (2,0) (2,1) -> 2nd Row
 - c1 c2

Initialization of 2D Array

Definition / Initialization

Ex.

```
int a[3][2] = {{10,20},{30,40},{50,60}};
```

or

```
int a[3][2] = {10,20,30,40,50,60};
```

a	a[][0]	a[][1]
a[0][]	10	20
a[1][]	30	40
a[2][]	50	60

Definition / Initialization

Ex.

```
int a[3][2] = {{10,20},{30,40},{50}};
```

or

```
int a[3][2] = {10,20,30,40,50};
```

a	a[][0]	a[][1]
a[0][]	10	20
a[1][]	30	40
a[2][]	50	0

Input & Output for 2D Array

```
int ar[3][2], i, j;  
// Input data  
for(i=0; i<3; i++)  
    for(j=0; j<2; j++)  
        scanf("%d", &ar[i][j]);  
  
//Output  
for(i=0; i<3; i++)  
    for(j=0; j<2; j++)  
        printf("%d\t", ar[i][j]);  
    printf("\n");
```

Input

10
20
30
40
50
60

Output

10 20
30 40
50 60

Program for 2D Array

```
#include<stdio.h>
void main()
{
int ar[3][2], i,j;
for(i=0;i<3;i++)
    for(j=0;j<2;j++)
        scanf("%d",&ar[i][j]);
for(i=0;i<3;i++)
    for(j=0;j<2;j++)
        printf("%d\t",ar[i][j]);
    printf("\n");
}
```

Input

10
20
30
40
50
60

Output

10 20
30 40
50 60

Program for 2D Matrix Addition

```
#include<stdio.h>
void main()
{
int a[3][2], b[3][2], c[3][2], i,j;
for(i=0;i<3;i++)
    for(j=0;j<2;j++)
        scanf("%d",&a[i][j]);
for(i=0;i<3;i++)
    for(j=0;j<2;j++)
        scanf("%d",&b[i][j]);
for(i=0;i<3;i++)
    for(j=0;j<2;j++)
        c[i][j]=a[i][j]+b[i][j];
```

```
for(i=0;i<3;i++)
    for(j=0;j<2;j++)
        printf("%d\t",c[i][j]);
    printf("\n");
}
```

Input

10	7
20	9
30	23
40	44
50	15
60	22

Output

17	29
53	84
65	82